Personalización de GitHup

Presentado por; Anny Sofia Beltrán García

Presentado a; Carlos Andrés Gómez Vasco

Medicina veterinaria

Universidad agraria de Colombia

UNIAGRARIA

Ciencia de datos

2026

CAPÍTULO 8. PERSONALIZACIÓN DE GIT

Introducción

Git permite adaptar su funcionamiento a las necesidades de cada usuario y proyecto. A través de diferentes opciones de configuración es posible establecer preferencias personales, definir reglas para determinados archivos y automatizar actividades que se realizan durante el desarrollo de un proyecto. La personalización de Git facilita el trabajo porque permite establecer configuraciones que se aplican a todos los repositorios o solamente a uno en particular. También permite utilizar herramientas automáticas para revisar los cambios y mantener determinadas reglas de organización dentro de un equipo

8.1 Configuración de Git

La configuración de Git permite modificar el comportamiento del programa según las preferencias del usuario. Para realizar estos ajustes se utiliza principalmente el comando git config, que permite establecer el nombre, el correo electrónico, el editor de texto, los colores y otras opciones

- **Git maneja tres niveles principales de configuración:**

1. Sistema: contiene configuraciones que se aplican a todos los usuarios del equipo
2. Global: establece preferencias para el usuario actual y se aplica a sus repositorios
3. Local: contiene configuraciones específicas para un repositorio

Cada nivel tiene una prioridad diferente. La configuración local tiene preferencia sobre la global, mientras que la global tiene preferencia sobre la del sistema

- **Algunos comandos utilizados para configurar Git son:**

- **git config --global user.name "Nombre del usuario**": establece el nombre que aparecerá en las confirmaciones
- **git config --global user.email "correo@ejemplo.com**": configura el correo electrónico del usuario
- **git config --global color.ui auto**: activa los colores automáticos en la terminal
- **git config --list:** muestra las configuraciones disponibles
- **git config --global core.editor "nombre_del_editor**": permite establecer el editor de texto que se utilizará para determinados mensajes

8.2 Atributos de Git

Los atributos de Git permiten establecer reglas específicas para determinados archivos o carpetas de un proyecto. Estas reglas se definen principalmente mediante un archivo llamado **.gitattributes**, que normalmente se ubica en la carpeta principal del repositorio. A través de los atributos se puede controlar cómo Git identifica, compara y administra ciertos archivos. También permiten establecer reglas para los finales de línea, indicar cómo deben tratarse determinados formatos y definir comportamientos especiales durante las fusiones. Un ejemplo de configuración es *** text=auto**, que permite a Git detectar automáticamente los archivos de texto y normalizar sus finales de línea. Otros atributos pueden utilizarse para indicar cómo deben compararse determinados archivos o qué estrategia se debe aplicar cuando se presentan conflictos durante una fusión. Los atributos son importantes porque permiten mantener un comportamiento uniforme dentro de un proyecto, especialmente cuando diferentes personas trabajan desde sistemas operativos distintos

8.3 Puntos de enganche en Git

Los puntos de enganche, también conocidos como hooks, son programas o scripts que Git ejecuta automáticamente cuando ocurre una acción determinada. Se utilizan para realizar comprobaciones y automatizar tareas durante el trabajo con un repositorio. Estos puntos de enganche se encuentran normalmente en la carpeta .git/hooks de cada repositorio. Git incluye algunos ejemplos que pueden utilizarse como referencia para crear automatizaciones propias

Entre los principales puntos de enganche se encuentran:

- **pre-commit:** se ejecuta antes de crear una confirmación y permite revisar los cambios que se van a guardar
- **commit-msg**: permite comprobar que el mensaje de una confirmación cumpla con un formato establecido
- **post-commit:** se ejecuta después de completar una confirmación
- **pre-push:** permite realizar comprobaciones antes de enviar los cambios a un repositorio remoto
- **post-merge**: se ejecuta después de completar una fusión

Los hooks ayudan a automatizar tareas repetitivas y a mantener reglas de calidad dentro de los proyectos. Sin embargo, algunos requieren conocimientos básicos de programación y deben configurarse correctamente para funcionar

8.4 Un ejemplo de implantación de una determinada política en Git

Git permite establecer políticas de trabajo para garantizar que los colaboradores cumplan determinadas reglas dentro de un proyecto. Estas políticas pueden aplicarse mediante configuraciones, puntos de enganche y permisos establecidos en el servidor

- **Para implementar estas políticas se pueden utilizar diferentes mecanismos:**
- Comprobar los mensajes de confirmación mediante un hook commit-msg
- Revisar los archivos antes de guardar cambios mediante un hook pre-commit
- Controlar los envíos al repositorio remoto mediante puntos de enganche del servidor
- Evitar que se realicen modificaciones no autorizadas en determinadas ramas
- Establecer reglas de colaboración para mantener organizado el proyecto

Las políticas permiten que todos los integrantes trabajen bajo las mismas condiciones. Esto ayuda a prevenir errores, mantener la calidad de los archivos y organizar las contribuciones dentro de un equipo. Es importante tener en cuenta que las comprobaciones realizadas únicamente en el computador de cada usuario pueden ser omitidas o modificadas. Por esta razón, las reglas que deben cumplirse obligatoriamente también deben verificarse en el servidor o mediante las herramientas de protección del repositorio

8.5 Ejercicio práctico: personalizar la configuración de Git

- **Procedimiento:**

1. Abrir Git Bash e ingresar a la carpeta de un repositorio de prueba mediante el comando **cd nombre_del_repositorio**
2. Consultar las configuraciones actuales utilizando **git config --list**
3. Configurar los colores de la terminal mediante **git config --global color.ui auto**
4. Crear un alias para consultar el estado del repositorio **utilizando git config --global alias.st status**
5. Ejecutar el comando **git st** para comprobar que el alias funciona correctamente
6. Crear un archivo llamado .gitattributes en la carpeta principal del repositorio

7. Escribir dentro del archivo la regla *** text=auto** para establecer el tratamiento automático de los finales de línea de los archivos de texto
8. Guardar el archivo y comprobar los cambios mediante **git status**
9. Agregar el archivo utilizando **git add .gitattributes**
10. Crear una confirmación mediante **git commit -m "Configuración de atributos de Git"**
11. Consultar nuevamente las configuraciones y el estado del repositorio para comprobar que los cambios se realizaron correctamente

Permite comprender cómo se pueden modificar las preferencias de Git y establecer reglas para organizar los archivos de un proyecto

Conclusiones

La personalización de Git permite adaptar el sistema de control de versiones a las necesidades de cada usuario y proyecto. Mediante el comando **git config** es posible establecer preferencias que facilitan el trabajo diario y permiten mantener una configuración organizada. Los atributos de Git ayudan a definir reglas para los archivos, mientras que los puntos de enganche permiten automatizar tareas y realizar comprobaciones durante diferentes etapas del trabajo. La implementación de políticas permite establecer condiciones para mantener la calidad y organización de los proyectos colaborativos. Estas herramientas hacen que Git sea más flexible y útil tanto en trabajos individuales como en equipos de desarrollo

Referencias bibliográficas

1. Chacon, S., & Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/es/v2
2. Git. (s. f.). Documentación de Git. https://git-scm.com/docs
3. GitHub. (s. f.). GitHub Docs. https://docs.github.com/es